Tutorial: Ambiente Virtual + VSCode + Streamlit

O Ciclo de Vida do Ambiente Virtual + VSCode + Streamlit

Passo 1:

Criar uma pasta com o Windows Explorer (**Aula3.2_ML-HelloStreamlit**)
Podemos escolher um local em nosso computador e criar uma pasta com o nome desejado, depois abrir o VS Code e clicar em “Abrir pasta” (Open folder) e localizar a pasta criada.

Passo 2: Criar o Ambiente virtual (Boa prática)

Agora é o momento de criar um ambiente virtual que irá conter todas as bibliotecas que vamos instalar para o projeto. Assim, evitamos qualquer inconsistência com bibliotecas já instaladas no computador.
No ambiente virtual conterá todas as bibliotecas que vamos instalar para o projeto. Dessa forma evitamos qualquer inconsistência com bibliotecas já instaladas no computador, além de garantir que as novas instalações utilizem as dependências específicas.

Navegue até a pasta do projeto e crie o ambiente. O nome deve ser claro para ser identificado depois. Para fazer isso, precisaremos abrir o terminal nessa pasta específica ("Aula3.2_ML-HelloStreamlit").

No menu superior, vamos clicar sobre o ícone de reticências (…) e ir até "Terminal > New Terminal". Para abrir o terminal, também podemos utilizar o atalho "Ctrl + Shift + '".

Na própria documentação do Streamlit podemos seguir o passo a passo para criar um ambiente virtual, acessando Ambiente Virtual para Streamlit(https://docs.streamlit.io/get-started/installation/command-line)

Com o VS Code aberto, abrir um terminal . Para criar o ambiente virtual, escreveremos o comando python -m venv .ve_Aula32_Hello,

Dentro da pasta(diretório) criar o ambiente virtual
cmd

```
python -m venv .ve_Aula32_Hello
```

Passo 3: Ativar o Ambiente virtual (Rode Streamlit a partir do Terminal do VSCode)

Para ativar o ambiente virtual, vamos retornar ao terminal e adicionar um novo comando. Com ele, vamos acessar a pasta "venv", a pasta "Scripts", e por fim, colocaremos /activate:

Agora é necessário "entrar" no ambiente para que as instalações não afetem o seu Windows global.
cmd

```
# Windows command prompt
.ve_Aula32_Hello\Scripts\activate

# Windows PowerShell
.ve_Aula32_Hello \Scripts\Activate.ps1
```

(Você verá o nome do ambiente entre parênteses no início da linha do terminal).

Feito isso, teremos um retorno com (.ve_Aula32_Hello) à esquerda do diretório no terminal, no início da linha. Esse é o nome do ambiente virtual, então agora, a partir de qualquer comando escrito no terminal, esse ambiente será utilizado.

Passo 4: Instalando as bibliotecas no Ambiente virtual

A seguir deve ser instalado os libs apenas o necessário para este projeto específico.

4.1) Instalação do Streamlit

A primeira biblioteca que vamos instalar será a biblioteca Streamlit. Para instalá-la, escreveremos o comando abaixo:

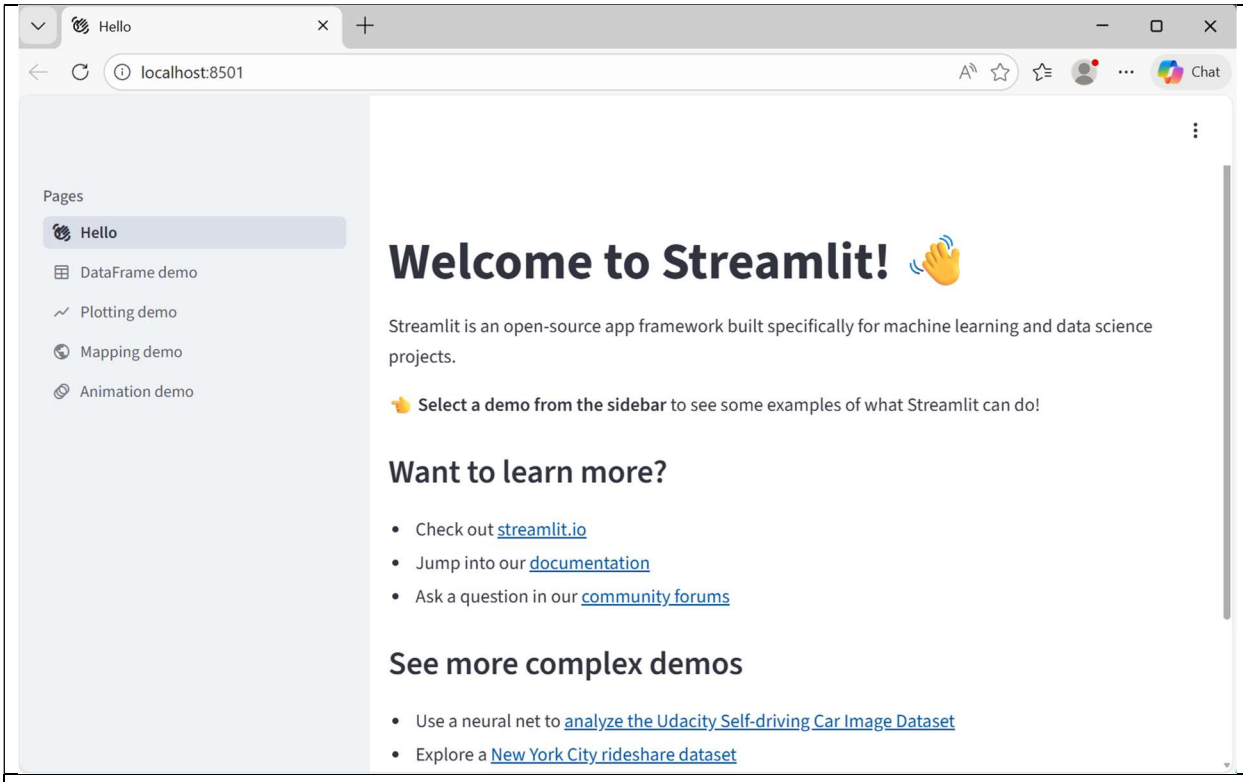
cmd

pip install streamlit

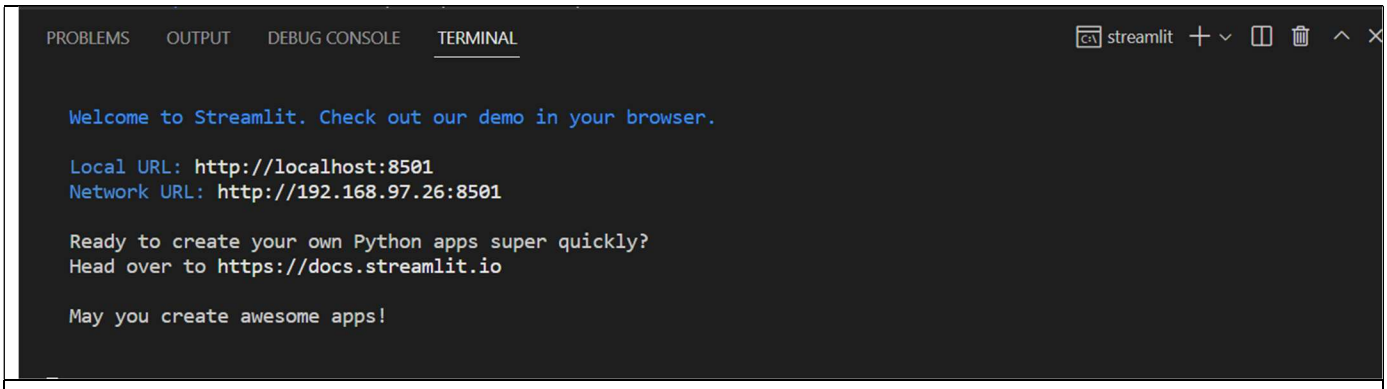
Para ter certeza que o streamlit está funcionando, digite:

cmd

streamlit hello



No terminal do VS Code aparecerá:



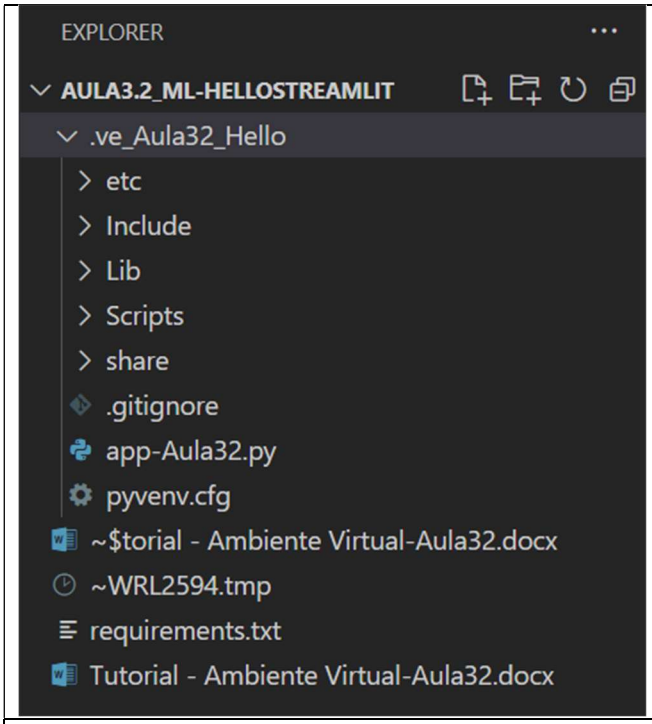
4.2 Demais bibliotecas (Requests, Pandas e Plotly)

Requests	pip install request	que permitirá a leitura dos dados a partir de uma API;
Pandas	pip install pandas	Pandas, usada para manipular os dados coletados da API
Plotly	pip install plotly	Plotly, que será utilizada para construir as visualizações no dashboard.

5) Criar e configurar o arquivo para o nosso Dashboard

5.1) Ciar o arquivo app.py

No VS Code, criar um arquivo com o nome app_hello.py:



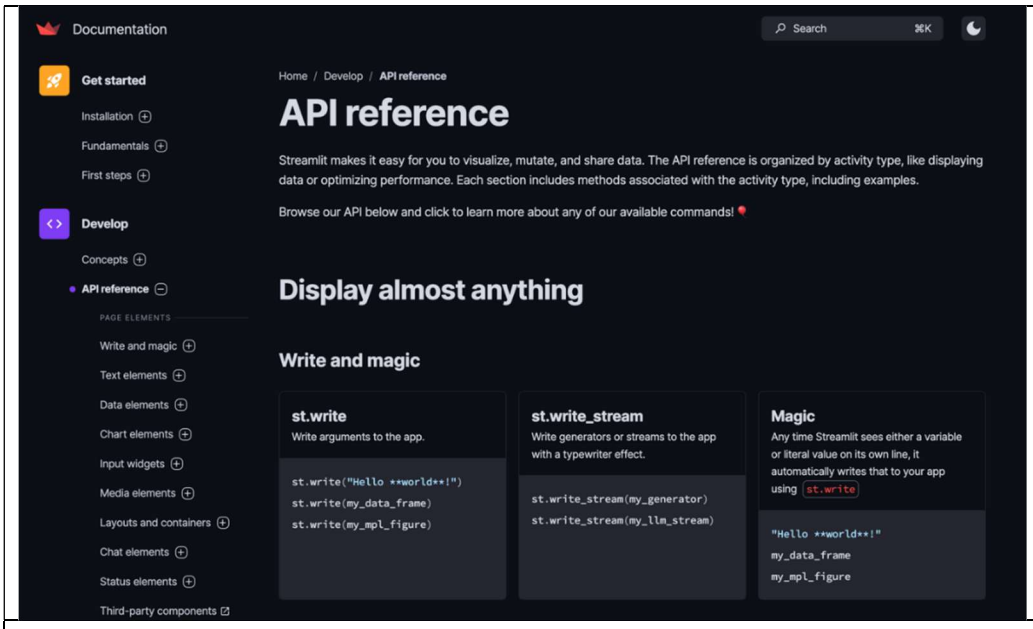
Mostrando a árvore do projeto com o arquivo app_hello.py

5.2) Configurar o arquivo

Dentro do arquivo app-Aula32.py, vamos importar as bibliotecas e dar um título do nosso Dashboard e colocar um emoji de café:

```
import streamlit as st
import requests
import pandas as pd
import plotly.express as px

st.title('DASHBOARD com :coffee:')

st.title('DASHBOARD DE VENDAS :shopping_trolley:')

url = 'https://labdados.com/produtos'
response = requests.get(url)
dados = pd.DataFrame.from_dict(response.json())

st.dataframe(dados)
```

A própria documentação do Streamlit é rica em detalhes sobre como podemos adicionar um título e outras personalizações do nosso projeto, acesse **aqui o tutorial oficial.**



Site da documentação do Streamlit com referência a API

6) Rodando nosso app

Para testar nosso projeto, vamos rodar o código abaixo:

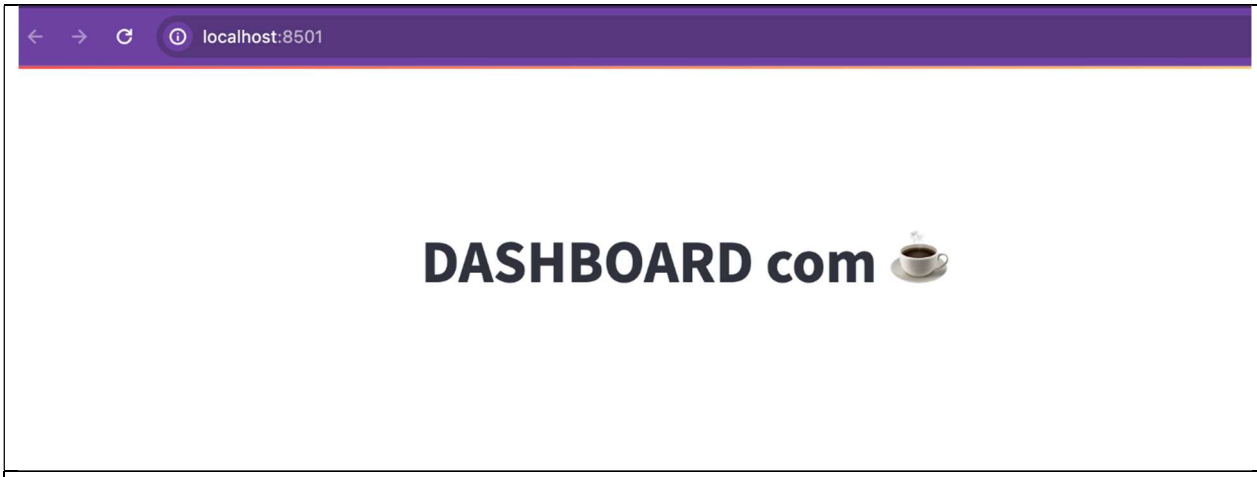
python -m streamlit run .ve_Aula32_Hello/app-Aula32.py

Ou

streamlit run .ve_Aula32_Hello/app-Aula32.py

Se fosse dado outro nome para o projeto, era só substituir app por esse outro nome do arquivo. Mas prefiro usar app pois futuramente para fazer o deploy no site do streamlit, vamos precisar que o arquivo esteja nomeado como app-Aula32.py

Após rodar no terminal o comando acima (pode demorar um pouco na primeira vez) seremos direcionados para o navegador exibindo a tela do nossa aplicação:



Tela da aplicação funcionando ao rodar python -m streamlit run .ve_Aula32_Hello/app-Aula32.py

7) Configurar o Git e GitHub

Inicie o repositório Git:
bash

git init

Crie um arquivo .gitignore para ignorar a pasta do ambiente virtual (essencial):

Crie um arquivo chamado .gitignore.
Adicione o seguinte dentro dele:

```text
.venv/
__pycache__/
.streamlit/
```

Crie seu arquivo de app (ex: app.py).
Faça o commit inicial:

bash

```
git add .
git commit -m "Commit inicial: configuracao ambiente virtual e streamlit"
```

Publicar no GitHub:

No VS Code, clique no ícone de Source Control (Controle de Versão) no menu lateral esquerdo.
Clique em "Publish to GitHub" (Publicar no GitHub) e escolha entre repositório público ou privado.

8) Como desativar o Ambiente Virtual (deactivate)

Quando terminar de trabalhar, para voltar ao Python do sistema:
No terminal do VS Code, digite:
Bash

```
.ve_Aula32_Hello\Scripts\deactivate.bat
```

Verifique: O prefixo (.venv) desaparecerá do terminal.

9) Gerando o arquivo no seu computador

Com o seu ambiente virtual ativado (aquele onde o Dashboard está funcionando), abra o terminal e execute o comando:

Bash

```
pip freeze > requirements.txt
```

Isso criará um arquivo muito mais detalhado, especificando a versão de cada dependência (ex: pandas==2.1.0). Isso é o ideal para garantir que nenhuma atualização futura das bibliotecas quebre o código do seu cliente.

O que esse comando faz?

- O pip freeze lista todas as bibliotecas instaladas e suas versões exatas.
- O símbolo > redireciona essa lista para dentro de um arquivo chamado requirements.txt.

10) O que enviar para o Cliente?

Para que o cliente recrie o ambiente com **um único comando**, você deve entregar uma pasta contendo:

1. app-Aula32.py (O script do Dashboard).
2. roi.csv (O banco de dados).
3. requirements.txt (O arquivo que acabamos de gerar).

11) Instrução para o Cliente (O "Único Comando")

Você deve instruir o seu cliente a abrir o terminal na pasta do projeto e seguir estes passos (que garantem o isolamento do computador dele):

1. **Criar o ambiente dele:** python -m venv .ve_Aula32_Hello
2. **Ativar o ambiente:** .ve_Aula32_Hello/app-Aula32.py\Scripts\activate (no Windows)
3. **O Comando Mágico:**

Bash

```
pip install -r requirements.txt
```

Dicas

1. Como interromper o Streamlit no VSCode?

Use o botão de Stop no terminal do VSCode ou pressione Ctrl+C.

2. Como abrir o app

Acesse: http://localhost:8501

3. Posso usar Streamlit no Google Colab?

Sim — instale Streamlit e use pyngrok para criar o túnel.

4. Por que usar ambientes virtuais?

Eles isolam suas dependências e evitam conflitos de versão.

5. Quais são as limitações comuns do Streamlit?

Customização de UI limitada, complexidade por ser stateless e não é ideal para apps de front-end em larga escala.

6. Para que Streamlit é usado?

Demos de ML, dashboards, ferramentas internas, exploração de dados e apps interativos rápidos.